

□ Les Décorateurs & Descripteurs en Python

Le support est une propriété de **Bechir Bejaoui**

1 □ Prérequis —les fonctions sont des objets

Avant de comprendre les décorateurs, il faut savoir une chose fondamentale en Python : **une fonction est un objet comme les autres**. On peut la stocker dans une variable, la passer en argument, la retourner.

```
def dire_bonjour():
    print("Bonjour !")

# Une fonction est un objet — on peut la stocker
action = dire_bonjour    # pas de () — on ne l'appelle pas, on la référence
action()                 # Bonjour !

# On peut la passer en argument à une autre fonction
def executer(fn):
    print("Avant...")
    fn()                 # appel de la fonction reçue
    print("Après.")

executer(dire_bonjour)

# On peut retourner une fonction depuis une autre fonction
def creer_salutation(nom):
    def saluer():
        print(f"Bonjour {nom} !")
    return saluer        # retourne la fonction, sans l'appeler

salut = creer_salutation("Bechir")
salut()                 # Bonjour Bechir !

Bonjour !
Avant...
Bonjour !
Après.
Bonjour Bechir !
```

- **action = dire_bonjour** sans les () : on stocke la **référence** à la fonction —pas son résultat
- **passer une fonction en argument** : c'est la base de la programmation fonctionnelle —et la base des décorateurs

- **retourner une fonction** : une fonction qui fabrique une autre fonction —c'est exactement ce que fait un décorateur
 - **fonction imbriquée** (saluer dans `creer_salutation`) : accède aux variables de la fonction englobante (nom) —c'est une **closure**
-

2 □ Les décorateurs —principe de base

Un **décorateur** est une fonction qui **enveloppe** une autre fonction pour lui ajouter un comportement, sans modifier son code. C'est du code qui s'exécute autour d'une fonction.

Avec la syntaxe @ :

```
import time

def chronometre(fn):
    def wrapper():
        debut = time.perf_counter()
        fn()
        duree = time.perf_counter() - debut
        print(f" □ Durée : {duree:.3f}s")
    return wrapper

@chronometre                                # ← équivaut à : calcul = chronometre(calcul)
def calcul():
    print(" Calcul en cours...")
    time.sleep(0.5)

calcul()

Calcul en cours...
□ Durée : 0.501s
```

- **@chronometre** est exactement équivalent à écrire `calcul = chronometre(calcul)` après la définition —c'est du sucre syntaxique
 - **la fonction calcul n'a pas changé** —c'est l'objet `calcul` qui pointe maintenant vers `wrapper`
 - **avantage** : le décorateur se lit d'un coup d'œil au-dessus de la fonction —l'intention est claire
-

3 □ Décorateur simple —anatomie complète

```
import time
```

```
# — Structure d'un décorateur —————  
#  
# def mon_decorateur(fn):      ← reçoit la fonction à décorer  
#     def wrapper():          ← la nouvelle fonction qui remplace fn  
#         # code AVANT  
#         fn()                ← appel de la fonction originale  
#         # code APRÈS  
#     return wrapper          ← retourne wrapper (pas wrapper() !)  
#
```

```
def journaliser(fn):  
    """Affiche un message avant et après chaque appel."""  
    def wrapper():  
        print(f" → Appel de '{fn.__name__}'")  
        fn()  
        print(f" ← Fin de '{fn.__name__}'")  
    return wrapper
```

```
@journaliser  
def envoyer_email():  
    print(" ✉ Email envoyé !")
```

```
@journaliser  
def sauvegarder():  
    print(" ✉ Sauvegarde effectuée !")
```

```
envoyer_email()  
print()  
sauvegarder()  
  
→ Appel de 'envoyer_email'  
✉ Email envoyé !  
← Fin de 'envoyer_email'
```

```
→ Appel de 'sauvegarder'  
✉ Sauvegarde effectuée !  
← Fin de 'sauvegarder'
```

- **fn.__name__** : accède au nom de la fonction originale depuis le wrapper
—fn est capturé par la closure
- **le même décorateur @journaliser** s'applique à n'importe quelle fonction sans la modifier —c'est la puissance des décorateurs

- **return wrapper** sans **()** : on retourne la fonction, pas son résultat — erreur classique d'ajouter les **()**

4 □ Décorateur avec arguments — *args, **kwargs

Un décorateur doit pouvoir s'appliquer à **n'importe quelle fonction**, quelle que soit sa signature. Pour ça, wrapper doit accepter et transmettre tous les arguments.

```
from functools import wraps

def verifier_entiers(fn):
    @wraps(fn)                                # conserve le nom et la doc de fn
    def wrapper(*args, **kwargs):           # accepte TOUS les arguments possibles
        for arg in args:
            if not isinstance(arg, (int, float)):
                raise TypeError(f"Argument invalide : {arg!r} n'est pas un nombre")
        return fn(*args, **kwargs)          # transmet tout à la fonction originale
    return wrapper

@verifier_entiers
def addition(a, b):
    """Additionne deux nombres."""
    return a + b

@verifier_entiers
def moyenne(a, b, c):
    return (a + b + c) / 3

print(addition(3, 4))                        # 7
print(moyenne(10, 20, 30))                  # 20.0

try:
    addition(3, "quatre")                    # ← déclenche la vérification
except TypeError as e:
    print(f"Erreur : {e}")

# Le nom et la doc sont préservés grâce à @wraps
print(f"Nom : {addition.__name__}")          # addition (pas 'wrapper')
print(f"Doc : {addition.__doc__}")           # Additionne deux nombres.
```

wraps sert à **préserver les informations de la fonction originale** lorsque l'on utilise un décorateur.

Sans précaution, Python considère alors que la fonction décorée **s'appelle**

wrapper et non plus addition ou moyenne

7

20.0

Erreur : Argument invalide : 'quatre' n'est pas un nombre

Nom : addition

Doc : Additionne deux nombres.

- ***args** : capture tous les arguments positionnels dans un tuple —(3, 4) pour addition(3, 4)
 - ****kwargs** : capture tous les arguments nommés dans un dict —{'n': 5} pour f(n=5)
 - **@wraps(fn)** : copie les métadonnées de fn vers wrapper —sans ça, addition.__name__ retournerait 'wrapper'
 - **return fn(*args, **kwargs)** : retransmet exactement les arguments reçus à la fonction originale —le décorateur est **transparent**
-

5 □ Décorateur avec paramètres

Parfois on veut **configurer** le décorateur lui-même : @repeter(3), @time-out(5). Il faut alors une fonction supplémentaire qui retourne le décorateur.

```
from functools import wraps
```

```
# Structure : 3 niveaux imbriqués
#
# def mon_decorateur(param):      ← niveau 1 : reçoit la configuration
#     def decorateur(fn):        ← niveau 2 : reçoit la fonction
#         def wrapper(*a, **k):  ← niveau 3 : s'exécute à chaque appel
#             ...
#             return wrapper
#         return decorateur
#     return decorateur
```

```
def repeter(n):
    """Répète l'exécution de la fonction n fois."""
    def decorateur(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            resultats = []
            for i in range(n):
                r = fn(*args, **kwargs)
                resultats.append(r)
            return resultats
        return wrapper
```

```

    return decorateur

@repeteur(3)                # ← repeteur(3) retourne un décorateur, qui décore saluer
def saluer(nom):
    print(f"  Bonjour {nom} !")
    return f"salut_{nom}"

resultats = saluer("Bechir")
print(f"Résultats : {resultats}")

    Bonjour Bechir !
    Bonjour Bechir !
    Bonjour Bechir !
Résultats : ['salut_Bechir', 'salut_Bechir', 'salut_Bechir']

```

Équivalent sans @ pour bien comprendre :

```

# @repeteur(3) est exactement équivalent à :
saluer = repeteur(3)(saluer)
#           _____ ← appel niveau 1 : retourne le décorateur
#           _____ ← appel niveau 2 : décore la fonction

```

- **repeteur(3)** : appel de niveau 1 —retourne décorateur avec n=3 capturé dans sa closure
- **decorateur(saluer)** : appel de niveau 2 —retourne wrapper avec fn=saluer capturé
- **wrapper(*args, **kwargs)** : appel de niveau 3 —s'exécute à chaque saluer(...) dans le programme

6 □ Décorateur de classe —@classmethod, @staticmethod, @property

Python fournit des décorateurs intégrés pour les classes. Ce sont les plus utilisés au quotidien.

class Produit:

```

nb_produits = 0    # attribut de classe

def __init__(self, nom: str, prix: float):
    self.nom = nom
    self._prix = prix
    Produit.nb_produits += 1

# @property : accès en lecture avec logique

```

```

@property
def prix(self):
    return f"{self._prix:.2f} TND"

# @prix.setter : validation à l'écriture
@prix.setter
def prix(self, valeur):
    if valeur < 0:
        raise ValueError("Le prix ne peut pas être négatif")
    self._prix = valeur

# @classmethod : méthode liée à la CLASSE (reçoit cls, pas self)
@classmethod
def creer_gratuit(cls, nom):
    return cls(nom, 0.0) # factory method

# @staticmethod : utilitaire lié à la classe, ni self ni cls
@staticmethod
def est_prix_valide(prix):
    return isinstance(prix, (int, float)) and prix >= 0

# Utilisation
p = Produit("Clavier", 99.9)
print(p.prix) # 99.90 TND ← @property

p.prix = 120
print(p.prix) # ← @prix.setter avec validation
# 120.00 TND

cadeau = Produit.creer_gratuit("Stylo") # ← @classmethod
print(cadeau.prix) # 0.00 TND

print(Produit.est_prix_valide(-5)) # False ← @staticmethod
print(Produit.nb_produits) # 2

99.90 TND
120.00 TND
0.00 TND
False
2

```

- **@property** : transforme une méthode en accès attribut —`p.prix` au lieu de `p.get_prix()` —plus pythonique
- **@prix.setter** : intercepte `p.prix = valeur` —permet la validation sans changer la syntaxe d'appel
- **@classmethod** : reçoit `cls` (la classe) —utile pour les **factory methods** qui créent des instances
- **@staticmethod** : ni `self` ni `cls` —fonction utilitaire rangée dans la classe

par logique, pas par besoin d'accès

7 □ Décorateurs empilés — l'ordre compte

On peut appliquer **plusieurs décorateurs** sur une même fonction. L'ordre d'application est **de bas en haut**, mais l'ordre d'exécution est **de haut en bas**.

```
from functools import wraps

def majuscules(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        print(" [majuscules] avant")
        resultat = fn(*args, **kwargs)
        print(" [majuscules] après")
        return resultat.upper() if isinstance(resultat, str) else resultat
    return wrapper

def exclamation(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        print(" [exclamation] avant")
        resultat = fn(*args, **kwargs)
        print(" [exclamation] après")
        return resultat + " !" if isinstance(resultat, str) else resultat
    return wrapper

@majuscules      # ← appliqué en 2ème (enveloppe exclamation(saluer))
@exclamation    # ← appliqué en 1er (enveloppe saluer directement)
def saluer(nom):
    return f"bonjour {nom}"

print(saluer("Bechir"))

[majuscules] avant
[exclamation] avant
[exclamation] après
[majuscules] après
BONJOUR BECHIR !
```

Visualisation :

```
@majuscules      ← couche externe — s'exécute en premier et en dernier
@exclamation    ← couche interne — s'exécute après majuscules
def saluer      ← fonction originale au centre
```


Exécution :

majuscules.avant → exclamation.avant → saluer() → exclamation.après → majuscules.après

- **@majuscules au-dessus de @exclamation** : équivaut à `saluer = majuscules(exclamation(saluer))`
 - **ordre d'application** : `exclamation` enveloppe `saluer` d'abord, puis `majuscules` enveloppe le tout
 - **ordre d'exécution** : `majuscules` → `exclamation` → `saluer` → `exclamation` → `majuscules`
 - **conclusion** : `"bonjour bechir" → + "!" → "bonjour bechir !" → .upper() → "BONJOUR BECHIR !"`
-

8 □ Les descripteurs —contrôler l'accès aux attributs

Un **descripteur** est une classe qui définit comment se comportent les opérations `get`, `set` et `delete` sur un attribut. C'est le mécanisme qui fait fonctionner `@property` en interne.

Le problème sans descripteur :

```
class Personne:
    def __init__(self, age):
        self.age = age  # aucune validation — on peut écrire n'importe quoi
```

```
p = Personne(25)
p.age = -10  # Python accepte — bug silencieux
p.age = "vieux"  # Python accepte aussi
print(p.age)  # "vieux" — incohérence
```

- **aucun contrôle** : n'importe quelle valeur est acceptée
 - **avec @property** : on peut valider, mais la logique est **dans la classe** —si on a 10 classes avec un âge, on duplique 10 fois
 - **avec un descripteur** : la logique est dans une **classe séparée réutilisable** —on l'applique partout
-

8.1 Data Descriptor —__get__, __set__, __delete__

Un **Data Descriptor** implémente au minimum `__get__` et `__set__`. Il a la priorité sur le `__dict__` de l'instance.

```
class EntierPositif:
    """Descripteur réutilisable : valide qu'un attribut est un entier > 0."""
```

```

def __set_name__(self, owner, name):
    # Appelé automatiquement quand le descripteur est assigné à une classe
    self.nom_public = name          # ex: "age"
    self.nom_stockage = f"_{name}"  # ex: "_age" — clé dans __dict__

def __get__(self, instance, owner):
    if instance is None:
        return self                # accès depuis la classe (Personne.age) → retourne le descripteur
    return instance.__dict__.get(self.nom_stockage, 0)

def __set__(self, instance, valeur):
    if not isinstance(valeur, int):
        raise TypeError(f"'{self.nom_public}' doit être un entier, reçu : {type(valeur)}")
    if valeur <= 0:
        raise ValueError(f"'{self.nom_public}' doit être > 0, reçu : {valeur}")
    instance.__dict__[self.nom_stockage] = valeur  # stockage dans __dict__ de l'instance

def __delete__(self, instance):
    print(f" Suppression de '{self.nom_public}'")
    instance.__dict__.pop(self.nom_stockage, None)

# Réutilisation du même descripteur pour plusieurs attributs
class Personne:
    age = EntierPositif()    # __set_name__ appelé → nom_public="age"
    taille = EntierPositif() # __set_name__ appelé → nom_public="taille"

    def __init__(self, nom, age, taille):
        self.nom = nom
        self.age = age      # ← déclenche EntierPositif.__set__
        self.taille = taille # ← déclenche EntierPositif.__set__

    def __repr__(self):
        return f"Personne({self.nom}, {self.age} ans, {self.taille} cm)"

p = Personne("Bechir", 35, 178)
print(p)                # Personne(Bechir, 35 ans, 178 cm)

p.age = 36               # ← __set__ déclenché — validation OK
print(p.age)             # 36 ← __get__ déclenché

try:
    p.age = -5           # ← __set__ déclenché — ValueError
except ValueError as e:

```

```

    print(f"Erreur : {e}")

try:
    p.taille = "grand"    # ← __set__ déclenché — TypeError
except TypeError as e:
    print(f"Erreur : {e}")

del p.age                # ← __delete__ déclenché

```

Personne(Bechir, 35 ans, 178 cm)
 36
 Erreur : 'age' doit être > 0, reçu : -5
 Erreur : 'taille' doit être un entier, reçu : str
 Suppression de 'age'

- **__set_name__** : méthode appelée **automatiquement** par Python quand le descripteur est assigné dans la classe — permet au descripteur de connaître son propre nom d'attribut
- **instance.__dict__[self.nom_stockage]** : stocke la valeur **dans le dict de l'instance** (pas dans le descripteur) — sinon tous les objets partageraient la même valeur !
- **if instance is None** : protège l'accès depuis la classe (Personne.age) — retourne le descripteur lui-même, pas une valeur
- **réutilisabilité** : EntierPositif s'applique à age et taille — zéro de duplication de logique

8.2 Non-Data Descriptor — __get__ uniquement

Un **Non-Data Descriptor** n'implémente que `__get__`. Il a une priorité **plus faible** que le `__dict__` de l'instance — peut être écrasé.

```

class MethodeCalc:
    """Descripteur non-data : ajoute une méthode calculée."""

    def __get__(self, instance, owner):
        if instance is None:
            return self
        # Retourne une fonction liée à l'instance
        def calculer():
            return instance.prix * 1.19    # prix TTC
        return calculer

class Produit:
    prix_ttc = MethodeCalc()    # non-data descriptor

```

```

def __init__(self, nom, prix):
    self.nom = nom
    self.prix = prix

p = Produit("Clavier", 100)
print(p.prix_ttc())          # 119.0 ← __get__ retourne la fonction, on l'appelle

# Un non-data descriptor peut être écrasé par l'instance
p.prix_ttc = "remplacé"     # écrit dans p.__dict__ directement
print(p.prix_ttc)           # "remplacé" ← l'instance gagne sur le descripteur

119.0
remplacé

```

Différence de priorité :

Accès à obj.attr :

Data Descriptor → priorité HAUTE (gagne sur __dict__)
(a __set__)

obj.__dict__[attr] → priorité MOYENNE

Non-Data Descriptor → priorité BASSE (perd contre __dict__)
(pas de __set__)

- **Non-Data Descriptor** : utilisé pour les **méthodes** en Python — obj.methode est un Non-Data Descriptor qui retourne une méthode liée
- **Data Descriptor** : utilisé pour les **attributs validés** — @property est un Data Descriptor intégré
- **@property est un Data Descriptor** : il définit __get__ ET __set__ (même si le setter lève AttributeError par défaut) → priorité haute

9 □ @property —descripteur intégré de Python

@property est la façon la plus pythonique d'ajouter de la logique sur un attribut sans changer la syntaxe d'accès.

class CompteBancaire:

```

def __init__(self, titulaire: str, solde: float = 0):
    self.titulaire = titulaire
    self._solde = solde          # convention : _ = "pas accès direct"
    self._historique = []

```

```

@property
def solde(self):
    """Lecture du solde — lecture seule depuis l'extérieur."""
    return self._solde

@property
def historique(self):
    return list(self._historique) # copie — l'original ne peut pas être modifié

def deposter(self, montant: float):
    if montant <= 0:
        raise ValueError("Montant doit être positif")
    self._solde += montant
    self._historique.append(f"+{montant}")

def retirer(self, montant: float):
    if montant > self._solde:
        raise ValueError("Solde insuffisant")
    self._solde -= montant
    self._historique.append(f"-{montant}")

compte = CompteBancaire("Bechir", 1000)
compte.deposer(500)
compte.retirer(200)

print(f"Solde      : {compte.solde} TND") # Lecture via @property
print(f"Historique: {compte.historique}")

try:
    compte.solde = 9999 # ← @property sans setter → AttributeError
except AttributeError as e:
    print(f"Lecture seule : {e}")

Solde      : 1300.0 TND
Historique: ['+500', '-200']
Lecture seule : can't set attribute

```

- **@property** sans setter = **attribut en lecture seule** — toute tentative d'écriture lève `AttributeError`
- **self._solde** : le vrai stockage avec `_` — convention "interne, ne pas accéder directement"
- **retourner `list(self._historique)`** : copie défensive — l'appelant ne peut pas modifier l'historique interne

□ Bonnes pratiques et guide de choix

```
from functools import wraps
```

```
# ☑ Pratique 1 : toujours @wraps pour préserver les métadonnées
```

```
def mon_decorateur(fn):  
    @wraps(fn) # sans ça : fn.__name__ = 'wrapper', fn.__doc__ = None  
    def wrapper(*args, **kwargs):  
        return fn(*args, **kwargs)  
    return wrapper
```

```
# ☑ Pratique 2 : toujours *args, **kwargs dans wrapper
```

```
# ☑ def wrapper(): → casse toute fonction avec arguments
```

```
# ☑ def wrapper(*a, **k): → transparent, fonctionne avec n'importe quelle signature
```

```
# ☑ Pratique 3 : descripteur → stocker dans instance.__dict__, pas dans self
```

```
class BonDescripteur:  
    def __set_name__(self, owner, name):  
        self.cle = f"_{name}" # clé unique par attribut  
  
    def __set__(self, instance, valeur):  
        instance.__dict__[self.cle] = valeur # ☑ dans l'instance  
  
    def __get__(self, instance, owner):  
        if instance is None: return self  
        return instance.__dict__.get(self.cle)
```

```
# ☑ Mauvais : stockage dans self du descripteur → tous les objets partagent la même valeur
```

```
class MauvaisDescripteur:  
    def __set__(self, instance, valeur):  
        self.valeur = valeur # ← stocké dans le descripteur, pas l'instance
```

Guide de choix :

Je veux ajouter un comportement autour d'une fonction ?

→ Décorateur @mon_decorateur

Je veux configurer ce comportement ?

→ Décorateur paramétré @mon_decorateur(option=True)

Je veux valider/contrôler UN attribut dans UNE classe ?

→ @property

Je veux la même validation pour le même attribut dans PLUSIEURS classes ?

→ Descripteur (classe séparée réutilisable)

`@wraps` → toujours dans un décorateur
`*args, **kwargs` → toujours dans `wrapper()`
`instance.__dict__` → toujours dans `__set__`
`if instance is None` → toujours dans `__get__`

□ Récapitulatif — les 5 points clés

Décorateur = fonction qui enveloppe une fonction · **@wraps** = préserve le nom et la doc · ***args, **kwargs** = rend le wrapper transparent · **@property** = attribut avec logique, syntaxe simple · **Descripteur** = `@property` réutilisable entre plusieurs classes

20 QUESTIONS INTELLIGENTES

Décorateurs & Descripteurs — Niveau Intermédiaire / Avancé

Chaque question contient :

- Énoncé
 - Explication structurée
 - Enchaînement démonstratif complet
 - Script commenté
 - Résultat observable
-

1 □ Pourquoi un décorateur casse parfois name et doc ?

Énoncé

Pourquoi une fonction décorée perd son nom et sa documentation ?

Explication

Quand on retourne `wrapper`, on remplace la fonction originale.
Sans précaution, les métadonnées sont perdues.

Solution : utiliser `functools.wraps`.

```
"""
Démonstration du problème :
- Sans wraps, Le nom devient 'wrapper'
"""
```

```
def decor(fn):
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)
    return wrapper
```

```
@decor
def mafonction():
    """Documentation originale"""
    print("Execution")

print("Nom:", mafonction.__name__)
print("Doc:", mafonction.__doc__)
```

Sortie observée :

Nom: wrapper
Doc: None

Correction :

```
"""
Solution correcte avec functools.wraps
"""
```

```
from functools import wraps

def decor(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)
    return wrapper
```

```
@decor
def mafonction():
    """Documentation originale"""
    print("Execution")
```



```
print("Nom:", mafonction.__name__)
print("Doc:", mafonction.__doc__)
```

2 □ Pourquoi un décorateur mal écrit casse les arguments ?

Énoncé

Pourquoi wrapper() sans *args, **kwargs est dangereux ?

```
"""
Erreur classique :
Le wrapper ne transmet pas les arguments.
"""

def decor(fn):
    def wrapper():
        return fn() # pas d'arguments transmis
    return wrapper

@decor
def addition(a, b):
    return a + b

try:
    print(addition(3, 4))
except TypeError as e:
    print("Erreur:", e)
```

Correction :

```
def decor(fn):
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)
    return wrapper
```

3 □ Pourquoi un décorateur sur méthode doit gérer self ?

Énoncé

Pourquoi un décorateur de méthode doit accepter self ?

```
"""  
Si wrapper oublie self, l'appel casse.  
"""
```

```
def decor(fn):  
    def wrapper(*args, **kwargs):  
        print("Avant")  
        return fn(*args, **kwargs)  
    return wrapper
```

```
class A:  
    @decor  
    def afficher(self):  
        print("OK")
```

```
a = A()  
a.afficher()
```

Observation : self est dans args automatiquement.

4 □ Pourquoi un décorateur paramétré a deux niveaux ?

Énoncé

Pourquoi il faut trois fonctions ?

```
"""  
Structure :  
decor(param) -> inner(fn) -> wrapper()  
"""
```

```
def decor(delai):
```

```

def inner(fn):
    def wrapper(*args, **kwargs):
        print("Avant")
        return fn(*args, **kwargs)
    return wrapper
return inner

@decor(3)
def test():
    print("OK")

test()

```

5 □ Pourquoi get reçoit instance et owner ?

Énoncé

À quoi sert owner ?

```

"""
- instance : objet courant
- owner : classe propriétaire
"""

class Descriptor:
    def __get__(self, instance, owner):
        print("Instance:", instance)
        print("Owner:", owner)
        return 42

class A:
    x = Descriptor()

a = A()
print(a.x)

```

6 □ Pourquoi un Data Descriptor a priorité sur dict ?

```
"""
Data Descriptor = définit __set__
Il prend priorité sur instance.__dict__
"""

class D:
    def __get__(self, instance, owner):
        return "Descriptor"

    def __set__(self, instance, value):
        instance.__dict__["x"] = value

class A:
    x = D()

a = A()
a.__dict__["x"] = "Direct"

print(a.x)  # priorité descriptor
```

7 □ Pourquoi un Non-Data Descriptor peut être écrasé ?

```
class D:
    def __get__(self, instance, owner):
        return "Descriptor"

class A:
    x = D()

a = A()
a.x = "Direct"

print(a.x)  # ici valeur instance
```

8 □ Pourquoi set peut provoquer récursion infinie ?

```
"""
Erreur fréquente :
instance.x = value dans __set__
Appelle encore __set__
"""

class D:
    def __set__(self, instance, value):
        instance.x = value # boucle infinie

class A:
    x = D()

Solution :
instance.__dict__["x"] = value
```

9 □ Pourquoi property est un Data Descriptor ?

```
class A:
    @property
    def x(self):
        return 10
```

```
a = A()
print(type(A.x))
```

Observation : property implémente **get** et **set**.

□ Pourquoi getattr est dangereux ?

```

"""
__getattr__ intercepte TOUT.
Une erreur bloque tout accès.
"""

class A:
    def __getattr__(self, name):
        print("Accès:", name)
        return super().__getattr__(name)

a = A()
print(a.__class__)

```